

.NET - Technical Interview Test

Project Overview

Your task is to develop a simple web application with an API and data layer using .NET C#, [ASP.NET](#) MVC, Web API, and a database or data store, while adhering to Clean Architecture principles and using Test-Driven Development (TDD) methodologies.

Your development should be driven by an informal user story that you will create, and which should be included in your presentation.

The application should allow users to create, read, update, and delete records from the data via the API endpoints. Additionally, you should create a user, login as the user, and ensure that the user information is stored in the data.

To showcase your ability to work with modern data storage systems you cannot use Entity Framework, Dapper, or Mediator to complete this assignment.

Requirements

Backend

● Database:

- Create a database or other data storage solution with at least one table/object/container to store data for the application and an additional one to store users.
- The table/object/container should have a unique identifier (primary key) and at least two other fields.

● API:

- Develop an [ASP.NET](#) Web API with endpoints that allow users to perform CRUD operations on the data.
- Each endpoint should have appropriate HTTP verbs, parameters, and return values.
- Additionally, a second API should include endpoints for user creation, user login, and authorized and non-authorized endpoints.

● Data layer:

- Develop a data access layer that interacts with the data and provides the necessary CRUD operations for the API endpoints.

● Business logic layer:

- Develop a business logic layer that includes all of the business rules and validation for the application.

- This layer should be independent of the data layer and the API.

- **Unit tests:**

- Write unit tests for all components of the application, including the data access layer, business logic layer, and API endpoints.

.NET - Technical Interview Exercise

Frontend

- While the main focus is on .NET, we need a decent level of full stack development skills, so:

- Please integrate the backend you just built with a frontend framework of your choice (React, Vue, etc.).

- Key criteria:

- The frontend should be responsive and user-friendly.

- Implement CRUD operations associated with the implemented use case.

- Structured code: Organize your components and state cleanly.

- **Submission Guidelines:**

- Include a README with setup instructions and any other documentation you deem necessary.

- The application should have seeded data / credentials for demo purposes.

Generative AI tools

Imagine you're tasked with generating a RESTful API for a simple task management system using your preferred language. The system should support the following functionality:

- Create, read, update, and delete tasks (CRUD)

- Each task has a title, description, status, and due_date

- Tasks are associated with a user (assume basic User model exists)

Instructions:

- Using your preferred GenAI coding tool (e.g., Cursor, Claude Code, Windsurf, GitHub Copilot, etc.), write the prompt you would use to generate the API scaffold or full implementation.

- Show the output code (or a representative sample of it).

- Describe how you:

- Validated the AI's suggestions

- Corrected or improved the output, if necessary

- Handled edge cases, authentication, or validations

Presentation and Code Review

You will be required to present your project to the technical interview panel. During the

presentation, you should explain your user story, design choices, the technical architecture, and demonstrate the functionality of the application. This will be done over Google Meets or Zoom and you will screen share either your GitHub repository or IDE.

.NET - Technical Interview Exercise

After the presentation, the interview panel will conduct a code review of your project. You will be asked to explain your coding decisions and answer any questions related to the code. The interview panel will evaluate your project based on the following criteria:

- Clean Architecture: Your architecture should adhere to Clean Architecture principles, including separation of concerns and independence of components.
- Application testing: Your project should have sufficient test coverage. Use of TDD is preferable.
- Code quality: Your code should be well-organized, readable, and adhere to best practices.
- Functionality: Your application should perform as expected in the requirements without errors or bugs. Optional but desired: no warnings in the browser console.
- Presentation: Your presentation should be clear, concise, and demonstrate a good understanding of the project and of the main backend and frontend best practices.
- GenAI tools: Your answers and presentation must show fluency with GenAI tools and prompt engineering, and critical thinking when evaluating AI-generated code.